

Ce code construit un modèle de régression logistique pour prédire si un client va résilier son abonnement (churn) ou non.

1. Chargement des données

```
url = "https://.../ChurnData.csv"  
churn_df = pd.read_csv(url)
```

Cette ligne charge le fichier CSV dans un DataFrame Pandas.

2. Sélection des variables

```
churn_df = churn_df[['tenure', 'age', 'address', 'income',  
                    'ed', 'employ', 'equip', 'churn']]
```

On conserve uniquement les colonnes utiles :

- tenure : Ancienneté du client
- age : Âge du client
- address : Nombre d'années à l'adresse actuelle
- income : Revenu
- ed : Niveau d'éducation
- employ : Ancienneté professionnelle
- equip : Nombre d'équipements/services utilisés
- churn : Variable cible (0 = reste client, 1 = résilie)

3. Conversion de la variable cible

```
churn_df['churn'] = churn_df['churn'].astype('int')
```

On convertit la colonne churn en entier afin qu'elle soit correctement interprétée par le modèle.

4. Création des variables explicatives et de la cible

```
X = np.asarray(churn_df[['tenure', 'age', 'address',  
                        'income', 'ed', 'employ', 'equip']])
```

```
y = np.asarray(churn_df['churn'])
```

- X contient les variables explicatives.
- y contient la variable à prédire (churn).

5. Normalisation des données

```
X_norm = StandardScaler().fit(X).transform(X)
```

La normalisation transforme chaque variable selon :

$$z = (x - \mu) / \sigma$$

où :

- μ = moyenne
- σ = écart-type

Après normalisation :

- moyenne ≈ 0
- écart-type ≈ 1

Cette étape est importante pour la régression logistique, car les variables sont alors sur la même échelle.

6. Division en ensemble d'entraînement et de test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_norm, y, test_size=0.2, random_state=4)
```

- 80 % des données servent à entraîner le modèle.
- 20 % servent à évaluer ses performances.

7. Entraînement du modèle

```
LR = LogisticRegression().fit(X_train, y_train)
```

Le modèle apprend la relation entre les variables explicatives et la probabilité qu'un client résilie.

$$P(\text{churn}=1) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)})$$

8. Prédiction des classes

```
yhat = LR.predict(X_test)
```

Le modèle prédit :

- 0 : le client reste

- 1 : le client résilie

9. Prédiction des probabilités

```
yhat_prob = LR.predict_proba(X_test)
```

Exemple :

```
[[0.90, 0.10],  
 [0.25, 0.75]]
```

Pour le premier client :

- 90 % de chance de rester
- 10 % de chance de résilier

Pour le second :

- 25 % de chance de rester
- 75 % de chance de résilier

10. Analyse des coefficients

```
coefficients = pd.Series(  
    LR.coef_[0],  
    index=churn_df.columns[:-1]  
)
```

```
coefficients.sort_values().plot(kind='barh')
```

Ce graphique montre l'influence de chaque variable sur le churn.

Interprétation :

- Coefficient positif → augmente le risque de résiliation.
- Coefficient négatif → diminue le risque de résiliation.

11. Évaluation avec la Log Loss

```
log_loss(y_test, yhat_prob)
```

La Log Loss mesure la qualité des probabilités prédites.

Interprétation :

- 0 : Parfait
- < 0,3 : Excellent
- 0,3 à 0,5 : Bon

- 0,5 à 0,7 : Moyen
- > 0,7 : Faible

Plus la valeur est proche de 0, meilleur est le modèle.

Conclusion

1. Chargement des données.
2. Sélection des variables.
3. Prétraitement (normalisation).
4. Séparation entraînement/test.
5. Entraînement d'une régression logistique.
6. Prédiction des classes et probabilités.
7. Analyse de l'importance des variables.
8. Évaluation du modèle avec la Log Loss.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
print("Accuracy :", accuracy_score(y_test, yhat))  
print("\nMatrice de confusion :")  
print(confusion_matrix(y_test, yhat))
```

```
print("\nRapport de classification :")  
print(classification_report(y_test, yhat))
```

5. Normalisation des données

</> Python



▶ Exécuter

```
x_norm = StandardScaler().fit(x).transform(x)
```

La normalisation transforme chaque variable selon :

$$z = \frac{x - \mu}{\sigma}$$

où :

- μ = moyenne,
- σ = écart-type.

Après normalisation :

- moyenne ≈ 0 ,
- écart-type ≈ 1 .

Cette étape est importante pour la régression logisti  car les variables sont alors sur la même échelle.

11. Évaluation avec la Log Loss

Python



Exécuter

```
log_loss(y_test, yhat_prob)
```

La Log Loss mesure la qualité des probabilités prédites :

$$LogLoss = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Interprétation

Valeur	Qualité du modèle
0	Parfait
< 0,3	Excellent
0,3 à 0,5	Bon
0,5 à 0,7	Moy 
> 0,7	Faible

Plus la valeur est proche de 0, meilleur est le modèle.